PyCaret

1. Información General y Arquitectura Base

- **¿Qué problema resuelve?** En el aprendizaje automático (*Machine Learning*) tradicional con librerías como `scikit-learn`, preparar los datos, probar distintos algoritmos, ajustar hiperparámetros y comparar métricas exige escribir cientos de líneas de código repetitivo (*boilerplate code*). PyCaret es una librería de bajo código (*low-code*) y aprendizaje automático automatizado (AutoML) en Python que resuelve este cuello de botella: reduce el tiempo de ciclo desde la hipótesis hasta la obtención de información (*insights*), permitiendo configurar experimentos, comparar docenas de modelos, optimizar el ganador y empaquetar un flujo de trabajo (*pipeline*) listo para producción en menos de veinte líneas de código.
    - **Comprobación en documentación oficial:** Página principal del proyecto (https://pycPyCaretaret.org/) y descripción oficial del paquete en PyPI (https://pypi.org/project/pycaret/).
    - **Fuentes adicionales:** Artículo académico sobre optimización de flujos de datos con AutoML (https://arxiv.org/pdf/2402.12916) y análisis técnico en *Towards Data Science* (https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/).
- **¿Quién lo desarrolla y mantiene?** Fue creado originalmente por **Moez Ali** (lanzado públicamente en abril de 2020) y es desarrollado y mantenido de forma colaborativa por él junto con una comunidad de contribuidores de código abierto bajo la organización `pycaret` en GitHub.
    - **Comprobación en documentación oficial:** Sección de citación y soporte en las preguntas frecuentes oficiales (http://pycaret.org/docs/resources/faqs/), metadatos de autoría en PyPI (https://pypi.org/project/pycaret/) y espejo del repositorio oficial de GitHub `[https://github.com/pycaret/pycaret](https://github.com/pycaret/pycaret)` (https://sourceforge.net/projects/pycaret.mirror/).
- **Licencia y costo** Es software **100 % gratuito y de código abierto** distribuido bajo la **Licencia MIT**. No tiene niveles de pago, licencias comerciales ni planes empresariales restrictivos (sin *vendor lock-in*).
    - **Comprobación en documentación oficial:** Sección de preguntas frecuentes (http://pycaret.org/docs/resources/faqs/) y clasificador de licencia en PyPI (https://pypi.org/project/pycaret/).
- **Versión actual y versiones de Python soportadas** *Hecho técnico importante:* El ecosistema de PyCaret atraviesa una transición entre su rama estable 3.x en PyPI y su nueva arquitectura 4.0 documentada en su portal principal:
    - **Rama 3.x (Estable en PyPI):** La versión estable publicada es la **3.3.2** (lanzada el 28 de abril de 2024). Soporta oficialmente **Python 3.9, 3.10 y 3.11**.
    - **Rama 4.0 (Nueva generación / Pre-lanzamientos en PyPI como `4.0.0a8`):** Reescrita sobre `scikit-learn 1.7+`, soporta **Python 3.11, 3.12 y 3.13** (eliminando soporte para Python 3.7–3.10 por fin de ciclo de vida).
    - **Comprobación en documentación oficial:** Guía de instalación de PyCaret 4.0 (http://pycaret.org/docs/getting-started/installation/), FAQ oficial

(http://pycaret.org/docs/resources/faqs/) e historial de versiones en PyPI (https://pypi.org/project/pycaret/).

- **¿Cómo se instala?** Se instala mediante el gestor de paquetes `pip` en la terminal o entorno virtual:
  - **Instalación base:** `pip install pycaret` (en la versión 4.0 instala un motor ligero con solo 13 dependencias directas, frente a las 65 de la rama 3.x).
  - **Extensiones opcionales (en 4.0):** `pip install "pycaret[dashboard]"` (interfaz visual y servidor FastAPI), `pip install "pycaret[explain]"` (librería SHAP para explicabilidad) o `pip install "pycaret[forecast]"` (adaptadores extra de `sktime` para series de tiempo).
  - **Extensiones completas (en 3.x):** `pip install pycaret[full]` o mediante contenedores Docker (`docker run -p 8888:8888 pycaret/slim` o `pycaret/full`).
  - **Desde código fuente (GitHub):** `git clone [https://github.com/pycaret/pycaret.git](https://github.com/pycaret/pycaret.git)`.
  - **Comprobación en documentación oficial:** Guía de instalación oficial (http://pycaret.org/docs/getting-started/installation/) y guía de instalación en PyPI (https://pypi.org/project/pycaret/).

2. Módulos y Funciones Clave

Módulos principales

En PyCaret cada tipo de problema se encapsula en un módulo independiente. En la versión 3.x se podían invocar mediante funciones sueltas (API funcional) o clases (API orientada a objetos, OOP); en la versión 4.0 la API funcional fue eliminada y se utiliza exclusivamente la API orientada a objetos:

Módulo	Ruta de importación y Clase OOP	Propósito	Fuente Oficial
Clasificación	`from pycaret.classification import ClassificationExperiment`	Predecir categorías discretas (ej. si un paciente tiene diabetes o no, o si un cliente cancelará su suscripción).	http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/latest/api/classification.html

Regresión	`from pycaret.regression import RegressionExperiment`	Predecir valores numéricos continuos (ej. el precio de una vivienda o nivel de ventas).	http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/stable/api/regression.html
Series de Tiempo	`from pycaret.time_series import TimeSeriesExperiment`	Pronosticar valores futuros ordenados temporalmente basándose en datos históricos y horizontes de predicción (`fh`).	http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/stable/modules.html
Clustering	`from pycaret.clustering import ClusteringExperiment`	Aprendizaje no supervisado para agrupar datos similares sin etiquetas previas (ej. segmentación con K-Means).	http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/stable/modules.html
Detección de Anomalías	`from pycaret.anomaly import AnomalyExperiment`	Aprendizaje no supervisado para identificar valores atípicos (*outliers*) o comportamientos inusuales en los datos.	http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/stable/modules.html

(Nota histórica: En versiones 2.x y 3.0 existían referencias a módulos de Procesamiento de Lenguaje Natural `pycaret.nlp` y Reglas de Asociación `pycaret.arules`, verificables en https://pycaret.readthedocs.io/en/stable/modules.html y https://anderfernandez.com/en/blog/pycaret-low-code-machine-learning-in-python/, pero el núcleo actual de PyCaret se concentra en los cinco módulos de la tabla anterior según https://pycaret.org/).

Funciones clave y qué automatiza cada una

1. **`setup()` (en 3.x) / `Experiment(...).fit(data)` (en 4.0):**
   - **Qué automatiza:** Inicializa el entorno de entrenamiento y construye automáticamente la tubería (*pipeline*) de transformación de datos. Ejecuta por defecto la división en entrenamiento y prueba (`train_size=0.7`, estratificada en clasificación y temporal en series de tiempo), imputación de valores faltantes (media en variables numéricas, moda en categóricas), codificación de variables categóricas (`OrdinalEncoder` y `LabelEncoder` para la variable objetivo), y opcionalmente normalización, eliminación de valores atípicos y selección de características.

    - **Comprobación oficial:** http://pycaret.org/docs/preprocessing/data-preparation/ y https://pycaret.readthedocs.io/en/latest/api/classification.html.
2. **`compare_models()`:**
    - **Qué automatiza:** Entrena y evalúa todos los estimadores disponibles en el catálogo de modelos mediante validación cruzada (*cross-validation*, por ejemplo, 10 particiones estratificadas). Calcula múltiples métricas de rendimiento de forma simultánea, muestra una tabla comparativa ordenada por la métrica elegida y devuelve el mejor modelo (o los `n_select` mejores).
    - **Comprobación oficial:** https://pycaret.readthedocs.io/en/latest/api/classification.html y análisis en https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/.
3. **`create_model()`:**
    - **Qué automatiza:** Entrena y evalúa un único modelo específico indicado por su identificador (ej. `'rf'` para Random Forest o `'lr'` para Regresión Logística) con sus hiperparámetros por defecto usando validación cruzada.
    - **Comprobación oficial:** http://pycaret.org/docs/getting-started/installation/ y https://pycaret.readthedocs.io/en/latest/api/classification.html.
4. **`tune_model()`:**
    - **Qué automatiza:** Busca y ajusta automáticamente los mejores hiperparámetros para un modelo dado dentro de un espacio de búsqueda predefinido (apoyándose en motores como `optuna` / estrategias de búsqueda), optimizando la métrica seleccionada mediante validación cruzada.
    - **Comprobación oficial:** https://pycaret.readthedocs.io/en/latest/api/classification.html y https://python.plainenglish.io/pycaret-simplified-mastering-low-code-machine-learning-e0277faed913.
5. **`ensemble_model()`, `blend_models()` y `stack_models()`:**
    - **Qué automatiza:** Automatizan la combinación de varios modelos individuales entrenados (métodos de ensamble, votación/*blending* o apilamiento/*stacking*) para mejorar la capacidad predictiva general.
    - **Comprobación oficial:** https://pycaret.readthedocs.io/en/latest/api/classification.html y https://pycaret.readthedocs.io/en/stable/api/regression.html.
6. **`plot_model()` y `evaluate_model()` / `pycaret.plots`:**
    - **Qué automatiza:** Generan visualizaciones diagnósticas del rendimiento del modelo (curvas ROC/AUC, matrices de confusión, importancia de variables, curvas de aprendizaje). En la versión 4.0, `evaluate_model()` fue reemplazado por la pantalla *Model Card* del dashboard y funciones modulares en `pycaret.plots` que devuelven gráficos interactivos de Plotly.
    - **Comprobación oficial:** http://pycaret.org/docs/resources/faqs/ y https://pycaret.readthedocs.io/en/latest/api/classification.html.
7. **`assign_model()` (en Clustering y Anomalías):**
    - **Qué automatiza:** Asigna a cada fila del conjunto de datos la etiqueta del clúster al que pertenece o su puntuación de anomalía tras entrenar un modelo no supervisado.
    - **Comprobación oficial:** https://python.plainenglish.io/pycaret-simplified-mastering-low-code-machine-learning-e0277faed913.

8. `predict_model()`:
    - **Qué automatiza:** Aplica toda la tubería de preprocesamiento y el modelo entrenado para generar predicciones sobre el conjunto de prueba (*hold-out test set*) o sobre datos nuevos no etiquetados (`data=new_data`), incluyendo intervalos de predicción en series de tiempo (`return_pred_int=True`).
    - **Comprobación oficial:** http://pycaret.org/docs/resources/cheat-sheet/ y https://pycaret.readthedocs.io/en/stable/api/regression.html.
9. `finalize_model()`:
    - **Qué automatiza:** Reentrena el modelo seleccionado utilizando el **100 % de los datos disponibles** (uniendo los conjuntos de entrenamiento y prueba) para aprovechar toda la información antes de llevar el modelo a producción.
    - **Comprobación oficial:** https://pycaret.readthedocs.io/en/stable/api/regression.html y https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/.
10. `save_model()`, `load_model()` **y despliegue (**`deploy_model()` **/ servidor HTTP):**
    - **Qué automatiza:** Guarda y restaura el flujo completo de transformación junto con el modelo entrenado en disco, o lo expone como un *endpoint* HTTP con estadísticas de latencia (p50/p95 en v4.0) y archivos de contenedor para producción.
    - **Comprobación oficial:** https://pycaret.org/, https://pycaret.readthedocs.io/en/latest/api/classification.html y https://arxiv.org/pdf/2402.12916.

3. Intervención en el Ciclo de Vida de los Datos

Dentro del ciclo de vida de un proyecto de ciencia de datos, PyCaret interviene directamente en **cuatro etapas centrales**:

1. **Preparación y preprocesamiento de datos:** Limpieza automatizada mediante imputación de nulos, codificación de variables categóricas, escalado/normalización, transformación de distribuciones, eliminación de valores atípicos (*outliers*) y selección de variables relevantes. (Fuente: http://pycaret.org/docs/preprocessing/data-preparation/).
2. **Modelado y experimentación:** Selección rápida de algoritmos candidatos (`compare_models`), entrenamiento con validación cruzada, ajuste de hiperparámetros (`tune_model`) y combinación de estimadores. (Fuente: https://arxiv.org/pdf/2402.12916).
3. **Evaluación e interpretación:** Cálculo de métricas de validación, análisis gráfico de errores (`plot_model`) y explicabilidad del modelo mediante valores SHAP (`pycaret[explain]`). (Fuente: http://pycaret.org/docs/getting-started/installation/ y https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/).
4. **Despliegue y puesta en producción (*Deployment*):** Reentrenamiento final con todos los datos (`finalize_model`), serialización del *pipeline* completo (`save_model`) y servicio mediante APIs/endpoints HTTP. (Fuente: https://pycaret.org/ y https://arxiv.org/pdf/2402.12916).

*Etapas donde **no** interviene:* No realiza la recolección o extracción inicial de datos crudos (consultas SQL, *web scraping*, ingesta de sensores) ni sustituye la comprensión del problema de negocio.

4. ¿Cuándo conviene usarla y cuándo no?

Cuándo SÍ conviene usarla

- **Prototipado rápido y líneas base (*baselines*):** Cuando profesionales o equipos de datos necesitan evaluar rápidamente una docena de algoritmos sobre datos tabulares o series de tiempo para encontrar qué familia de modelos funciona mejor. (Fuente: https://pypi.org/project/pycaret/ y https://www.researchgate.net/publication/391344604_Simplifying_Machine_Learning_with_PyCaret_A_Low-Code_Approach_for_Beginners_and_Experts).
- **Entornos educativos y "Citizen Data Scientists":** Ideal para estudiantes, principiantes o analistas de negocio que conocen los fundamentos teóricos pero buscan construir flujos completos sin dominar la programación avanzada en `scikit-learn`. (Fuente: https://pypi.org/project/pycaret/).
- **Aumento de productividad en tareas estándar:** Proyectos de clasificación, regresión (como diagnóstico médico o predicción de abandono de clientes), clustering o detección de anomalías donde un *pipeline* estándar de `scikit-learn` es suficiente. (Fuente: https://www.researchgate.net/publication/371252937_A_novel_method_for_diabetes_classification_and_prediction_with_Pycaret).

Cuándo NO conviene usarla

- **Reglas de preprocesamiento altamente personalizadas por columna:** La propia documentación oficial de PyCaret 4.0 advierte que la librería es "obstinada con sus valores por defecto" (*opinionated about defaults*); por ejemplo, imputa todas las columnas numéricas con la media y las categóricas con la moda. Si necesitas imputar la mediana en una columna específica y aplicar reglas distintas en otras, PyCaret no ofrece banderas para ello y exige que lo programes manualmente con `pandas` o `scikit-learn` antes de pasarle los datos (o pasando `preprocess=False`). (Fuente: http://pycaret.org/docs/preprocessing/data-preparation/ y http://pycaret.org/docs/resources/faqs/).
- **Mantenimiento de sistemas heredados con mezcla de versiones:** Si un proyecto ya está escrito en PyCaret 3.x y no puede refactorizarse, no conviene actualizar a 4.0 en el mismo código porque no son retrocompatibles y no se soporta mezclar ambas APIs. (Fuente: http://pycaret.org/docs/resources/faqs/).
- **Problemas fuera de su alcance arquitectónico (como Deep Learning de visión o audio crudo):** Aunque la literatura menciona que AutoML inspira mejoras en flujos de datos, los módulos de PyCaret están diseñados sobre estimadores de `scikit-learn` y `sktime` para datos estructurados/tabulares y series temporales, no para entrenar redes neuronales profundas sobre imágenes o video directamente. (Fuente: https://pycaret.org/ y https://arxiv.org/pdf/2402.12916).

5. Ventajas, Limitaciones y Riesgos Críticos

Ventajas demostradas

- **Ahorro drástico de código y tiempo:** Permite comparar hasta 18 clasificadores bajo 6 métricas con validación cruzada de 10 particiones en una sola línea de código. (Fuente: https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/).
- **Tuberías nativas sin envoltorios propietarios (en v4.0):** A diferencia de versiones anteriores, PyCaret 4.0 expone objetos `Pipeline` nativos de `scikit-learn 1.7+`, evitando bloqueos tecnológicos (*no vendor lock-in*). (Fuente: https://pycaret.org/).
- **Soporte para aceleración por hardware:** Permite activar entrenamiento en GPU (`use_gpu=True`) para algoritmos compatibles (XGBoost, LightGBM, CatBoost, cuML) y optimizaciones de Intel (`scikit-learn-intelex`). (Fuente: http://pycaret.org/docs/getting-started/installation/ y https://pypi.org/project/pycaret/).

Limitaciones y Riesgos

1. **Reproducibilidad:**
    - *El riesgo:* La división aleatoria de datos, el muestreo en validación cruzada y la inicialización de ciertos algoritmos generan resultados distintos en cada ejecución si no se controlan.
    - *Cómo se gestiona:* PyCaret exige fijar el parámetro `session_id` (por ejemplo, `session_id=42` o `session_id=123`, equivalente a `random_state` en `scikit-learn`) al inicializar el experimento. La documentación garantiza que el mismo `session_id` con los mismos datos produce exactamente la misma partición reproducible. Sin embargo, cambiar de versión de librería o entorno puede alterar resultados.
    - *Fuente:* http://pycaret.org/docs/preprocessing/data-preparation/ y https://python.plainenglish.io/pycaret-simplified-mastering-low-code-machine-learning-e0277faed913.
2. **Dependencia de versiones y ruptura de compatibilidad (*Breaking Changes*):**
    - *El riesgo:* Históricamente, PyCaret 3.x arrastraba **65 dependencias directas**, lo que generaba conflictos frecuentes de instalación y obligaba a restringir versiones específicas de Python o usar contenedores Docker. Además, el salto a PyCaret 4.0 introdujo una **ruptura total de retrocompatibilidad**: eliminó la API funcional (`setup()`, `compare_models()` como funciones sueltas) y exige `scikit-learn >= 1.7` y `Python >= 3.11`. El código escrito por estudiantes siguiendo tutoriales de la versión 2.x o 3.x fallará en la versión 4.0 si no se adapta a clases como `ClassificationExperiment`.
    - *Fuente:* http://pycaret.org/docs/getting-started/installation/, http://pycaret.org/docs/resources/faqs/ y https://anderfernandez.com/en/blog/pycaret-low-code-machine-learning-in-python/.
3. **Efecto "Caja Negra" y errores silenciosos de configuración:**
    - *El riesgo:* Al automatizar todo el flujo en un par de comandos, un estudiante sin bases teóricas puede entrenar modelos sin comprender qué transformaciones sufrieron sus datos o cómo operan los algoritmos internamente. Asimismo, la propia documentación oficial reconoce que en la versión 3.x la función `setup()` aceptaba cerca de 100 argumentos opcionales donde muchos no estaban documentados, varios estaban rotos y los errores tipográficos se ignoraban silenciosamente (en la versión 4.0 se corrigió lanzando un `ConfigurationError` explícito). Para mitigar la opacidad de los modelos predictivos, se requiere instalar el módulo extra de

explicabilidad (`pycaret[explain]` con SHAP) y analizar las gráficas de diagnóstico.
    - *Fuente:* http://pycaret.org/docs/resources/faqs/, http://pycaret.org/docs/getting-started/installation/ y https://www.researchgate.net/publication/391344604_Simplifying_Machine_Learning_with_PyCaret_A_Low-Code_Approach_for_Beginners_and_Experts.
4. **Sesgo (*Bias*) e impacto de las decisiones automatizadas:**
    - *El riesgo:* La automatización de decisiones predeterminadas (como imputar siempre con la media aritmética o la moda en `Data preparation`, o codificar ordinalmente todas las variables categóricas de forma automática) puede distorsionar la distribución real de los datos si existen asimetrías severas o valores faltantes no aleatorios. Además, si el usuario desactiva el preprocesamiento automático (`preprocess=False`) y no aplica la disciplina estricta de ajustar transformadores solo en entrenamiento (`fit` en *train*, `transform` en *test*), incurre en fuga de datos (*data leakage*).
    - *Nota sobre equidad algorítmica (Fairness):* **No se encontró fuente** en los fragmentos recuperados de la documentación actual de PyCaret 4.0 (`pycaret.org/docs`) que documente una función activa para auditar sesgos demográficos o equidad algorítmica (como `check_fairness` de versiones previas).
    - *Fuente:* http://pycaret.org/docs/preprocessing/data-preparation/ y http://pycaret.org/docs/resources/faqs/.

6. Referencias Bibliográficas (Formato APA 7)

- Ali, M., & Contributors. (2024). *pycaret: An open source, low-code machine learning library in Python* (Versión 3.3.2) [Software de computador]. Python Package Index (PyPI). https://pypi.org/project/pycaret/
- Ali, M., & Contributors. (2026). *PyCaret: Low-code machine learning for Python* (Versión 4.0) [Documentación oficial y código fuente]. GitHub / PyCaret. https://pycaret.org/
- Fernández, A. (s.f.). *PyCaret: Low-code machine learning in Python*. Ander Fernández Jauregui Blog. https://anderfernandez.com/en/blog/pycaret-low-code-machine-learning-in-python/
- PyCaret Developers. (2023). *PyCaret 3.0.4 documentation*. Read the Docs. https://pycaret.readthedocs.io/en/stable/modules.html
- Whig, P., Velu, A., Reddy, R., & Sharma, P. (2023). A novel method for diabetes classification and prediction with Pycaret. *Microsystem Technologies*, *29*(10), 1479–1487. https://www.researchgate.net/publication/371252937_A_novel_method_for_diabetes_classification_and_prediction_with_Pycaret
- Wu, J., & coautores. (2024). *Data pipeline training: Integrating AutoML to optimize the data flow of machine learning models* (arXiv:2402.12916). arXiv. https://arxiv.org/abs/2402.12916
- Yıldırım, S. (2020, 13 de julio). *Machine learning made easy by PyCaret*. Towards Data Science. https://towardsdatascience.com/machine-learning-made-easy-by-pycaret-5be22394b1ac/